PROYECTO FINAL:

Agenda

ASIGNATURA: Sistemas Web I

FECHA: 9 de diciembre de 2025

ALUMNO: Álvaro Sanz Cortés

TRABAJO: Memoria

ÍNDICE

Tabla de contenido

1. Descripción del proyecto 3

2. Sitemap 3

3. Histórico de cambios 3

4. Diagrama de casos de uso 4

5. Arquitectura 5

6. Requisitos 5

6.1. Requisitos cumplidos y No cumplidos 5

6.2. Requisitos Funcionales 6

6.3. Requisitos No Funcionales 7

7. Enlace al repositorio 8

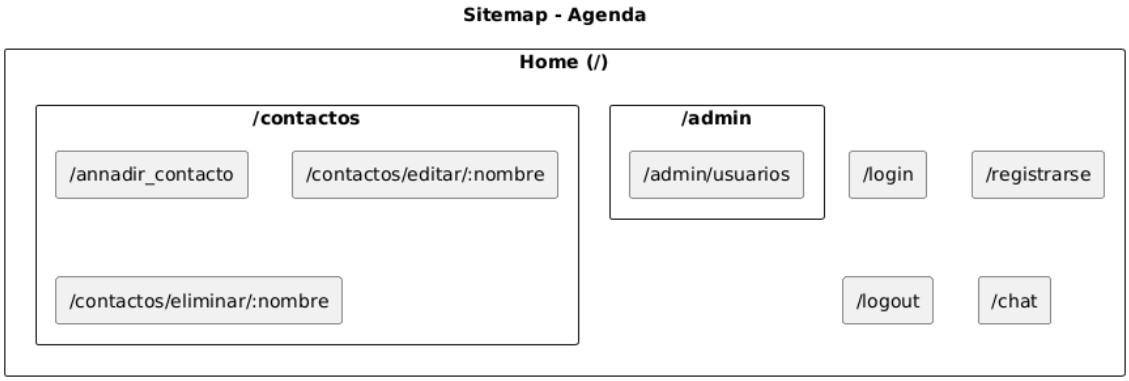
8. Manual de Uso 8

1. Descripción del proyecto

Este proyecto consiste en el desarrollo de una **aplicación web de agenda personal**, que permite gestionar contactos y comunicarse mediante un sistema de chat en tiempo real.
La plataforma está construida con **Node.js, Express, Socket.io y EJS**, utilizando una estructura cliente-servidor y con almacenamiento persistente de información.

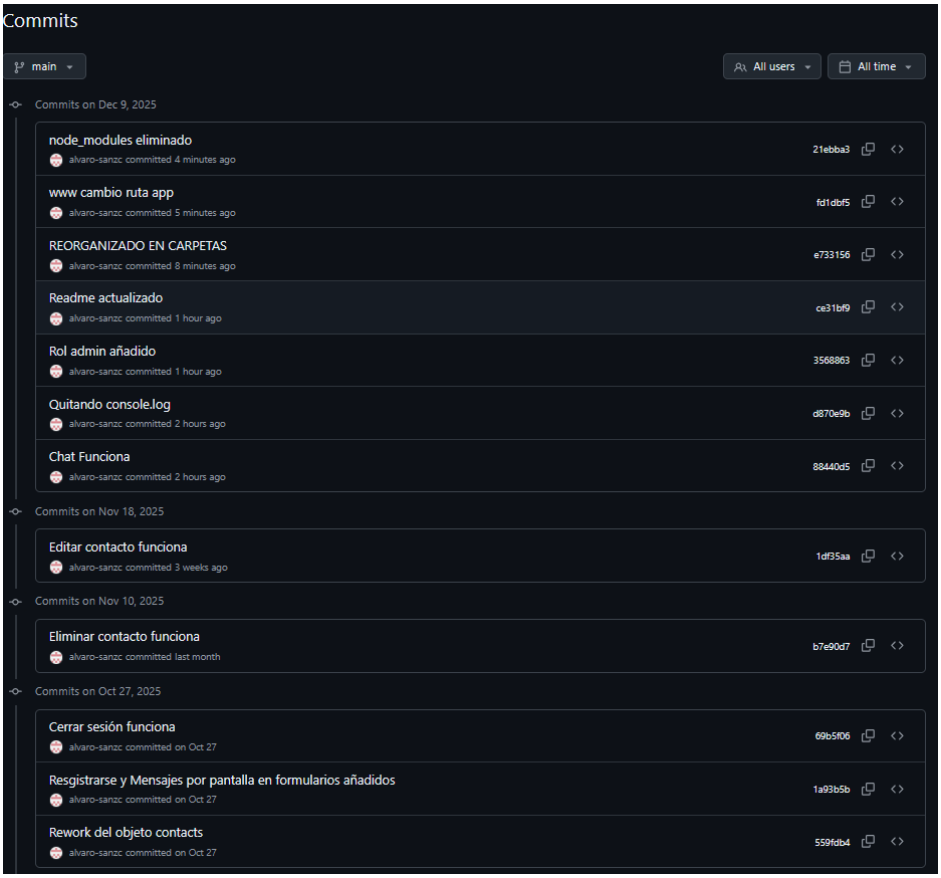
El objetivo principal de la aplicación es ofrecer una agenda funcional donde los usuarios puedan **registrarse, iniciar sesión, gestionar contactos y comunicarse con otros usuarios conectados**, incluyendo un sistema de roles con permisos diferenciados.

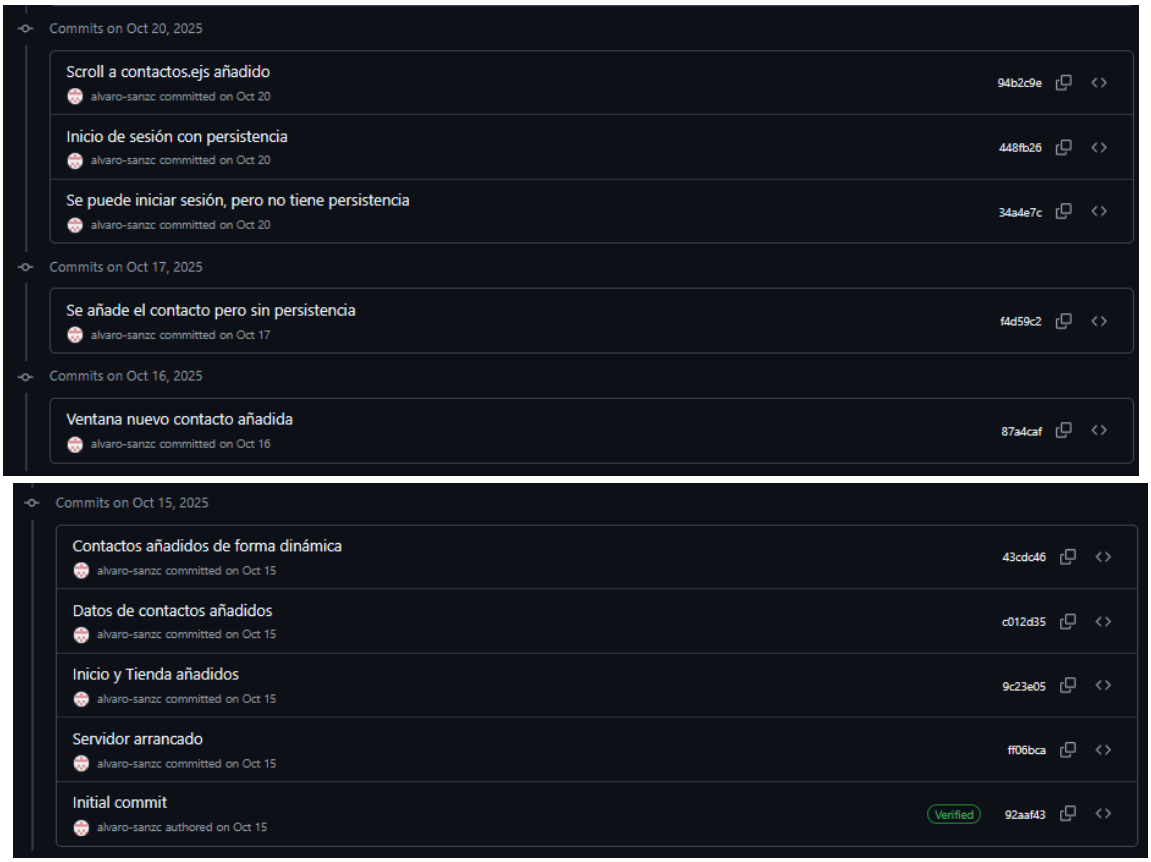
2. Sitemap



3. Histórico de cambios

El control de versiones y el registro de cambios del proyecto se ha gestionado con **Git y GitHub**, permitiendo llevar un seguimiento detallado del progreso, nuevas características implementadas, corrección de errores y mejoras en el código.

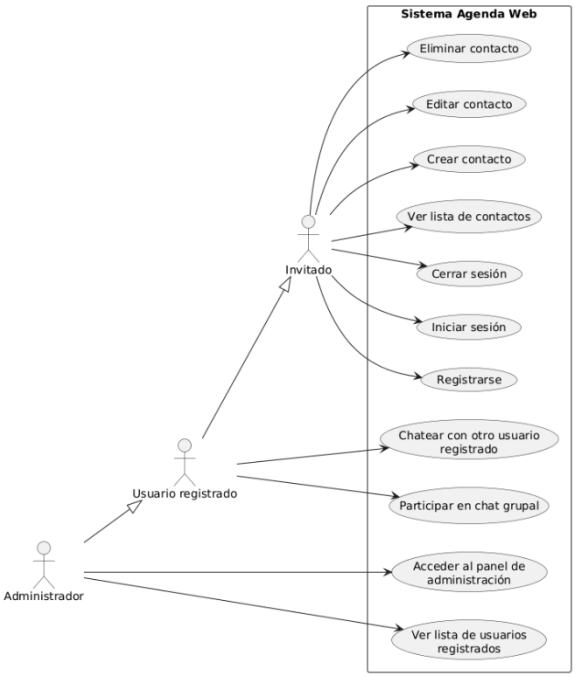




4. Diagrama de casos de uso

El diagrama de casos de uso permite representar de forma visual las funcionalidades principales del sistema y la interacción que los diferentes tipos de usuarios tienen con él. Este diagrama muestra qué acciones puede realizar cada rol dentro de la aplicación, diferenciando entre invitado, usuario registrado y administrador.

- El *invitado* tiene acceso a las funciones básicas del sistema, como registrarse, iniciar sesión, cerrar sesión y gestionar contactos.
- El *usuario registrado* amplía sus permisos pudiendo acceder al chat, en el puede acceder a un chat global en el que pueden escribir todos los usuarios registrados
- Finalmente, el *administrador* dispone de acceso total al sistema, incluyendo el panel de administración y la visualización de usuarios registrados.

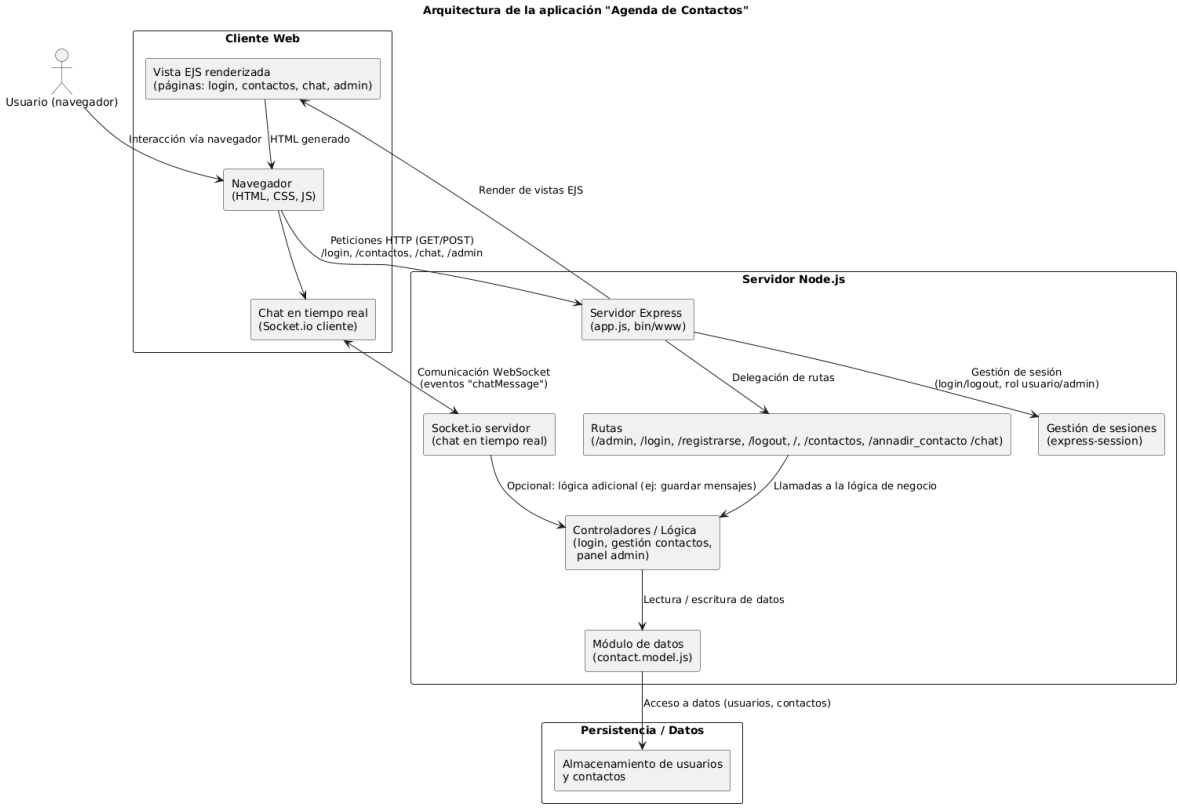


5. Arquitectura

En este proyecto se ha utilizado una **arquitectura web cliente-servidor**, donde el cliente (navegador) interactúa con el servidor a través de peticiones HTTP y, en el caso del chat, mediante comunicación en tiempo real con WebSockets.

El servidor ha sido desarrollado con **Node.js y Express**, gestionando las rutas de la aplicación, la autenticación de usuarios, la lógica de negocio y el acceso a los datos. La interfaz de usuario se genera mediante **vistas EJS**, mientras que los recursos estáticos (CSS y JavaScript) se sirven desde la carpeta public. Para el almacenamiento de datos se ha empleado un módulo de persistencia que gestiona usuarios y contactos, y para las funciones en tiempo real se ha utilizado **Socket.io**, permitiendo comunicación bidireccional entre los clientes conectados.

Esta arquitectura modular facilita el mantenimiento, la escalabilidad y la comprensión del proyecto. Cada componente cumple una responsabilidad concreta, separando la gestión de vistas, lógica, datos y comunicación en tiempo real.



6. Requisitos

6.1. Requisitos cumplidos y No cumplidos

Tras el desarrollo del sistema, se puede concluir que **todos los requisitos para la aplicación han sido cumplidos**.

6.2. Requisitos Funcionales

1. **RF01 – Registro de usuario**
   El sistema debe permitir que un invitado se registre proporcionando un nombre de usuario y una contraseña válidos.

2. **RF02 – Inicio de sesión**
   El sistema debe permitir que un usuario registrado inicie sesión introduciendo su nombre de usuario y contraseña.

3. **RF03 – Cierre de sesión**
   El sistema debe permitir que un usuario autenticado (registrado o administrador) cierre su sesión de forma segura.

4. **RF04 – Gestión de contactos**
   El sistema debe permitir crear nuevos contactos en la agenda indicando, como mínimo, nombre y teléfono

5. **RF05 – Gestión de contactos: edición**
   El sistema debe permitir modificar los datos de un contacto existente.

6. **RF06 – Gestión de contactos: eliminación**
   El sistema debe permitir eliminar contactos de la agenda.

7. **RF07 – Visualización de contactos**
   El sistema debe mostrar la lista de contactos, incluyendo su información básica.

8. **RF-08 – Chat entre usuarios registrados**
   El sistema debe permitir que dos usuarios registrados se envíen mensajes a través de un chat en tiempo real.

9. **RF-09 – Chat grupal**
   El sistema debe permitir que más de dos usuarios registrados participen simultáneamente en una misma sala de chat.

10. **RF-10 – Roles de usuario**
    El sistema debe diferenciar, como mínimo, entre los roles de **invitado**, **usuario registrado** y **administrador**.

11. **RF11 – Permisos de invitado**
    El sistema debe permitir que el invitado pueda acceder a la aplicación, visualizar la agenda y registrarse / iniciar sesión.
    (Ajusta aquí si en tu implementación el invitado también puede añadir/editar/eliminar contactos.)

12. **RF12 – Permisos de administrador**
    El sistema debe permitir que el administrador pueda realizar todas las acciones del usuario registrado y, adicionalmente, acceder a un panel de administración.

13. **RF13 – Panel de administración**
    El sistema debe disponer de un panel de administración accesible solo para usuarios con rol administrador.

14. **RF14 – Visualización de usuarios registrados**
    El sistema debe permitir que, desde el panel de administración, se muestre la lista de usuarios registrados.

6.3. Requisitos No Funcionales

1. **RNF01 – Arquitectura cliente-servidor**
   La aplicación debe seguir una arquitectura cliente-servidor basada en Node.js y Express, con vistas generadas en el servidor y consumidas en el navegador.

2. **RNF02 – Usabilidad**
   La interfaz de usuario debe ser sencilla e intuitiva, permitiendo acceder a las principales funciones (agenda, login, chat, administración, etc.) de forma clara.

3. **RNF03 – Rendimiento**
   El sistema debe ser capaz de gestionar múltiples usuarios conectados al chat en tiempo real sin tiempos de respuesta excesivos.

4. **RNF04 – Tiempo real**
   La comunicación del chat debe realizarse mediante tecnología de `WebSockets` (Socket.io), permitiendo el envío y recepción de mensajes sin recargar la página.

5. **RNF05 – Seguridad de sesión**
   Las sesiones de usuario deben gestionarse en el servidor mediante un mecanismo de sesión (por ejemplo, `express-session`) y una clave secreta.

6. **RNF06 – Control de acceso por roles**
   El sistema debe restringir el acceso a determinadas rutas y funcionalidades según el rol del usuario (invitado, usuario registrado y administrador).

7. **RNF07 – Mantenibilidad**
   El código debe estar organizado en módulos (rutas, vistas, modelos, middlewares) para facilitar su mantenimiento y futuras ampliaciones.

8. **RNF08 – Portabilidad**
   La aplicación debe poder ejecutarse en cualquier entorno que disponga de Node.js y un navegador web moderno.

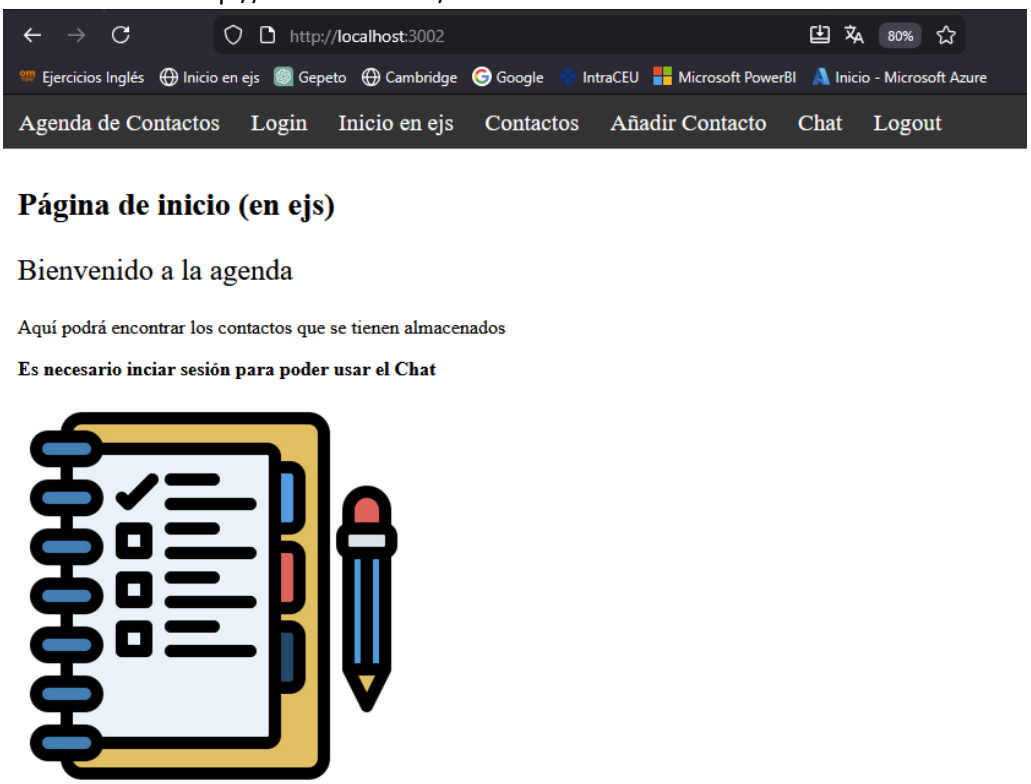
9. **RNF09 – Registro de cambios**
   El desarrollo del proyecto debe estar versionado mediante Git y alojado en un repositorio (GitHub), permitiendo el seguimiento del histórico de cambios.
10. **RNF-10 – Consistencia visual**
    Las diferentes secciones (login, agenda, chat, panel admin) deben mantener una apariencia visual coherente mediante el uso de hojas de estilo compartidas.

7. Enlace al repositorio

Este es el repositorio: https://github.com/alvaro-sanzc/Agenda

8. Manual de Uso

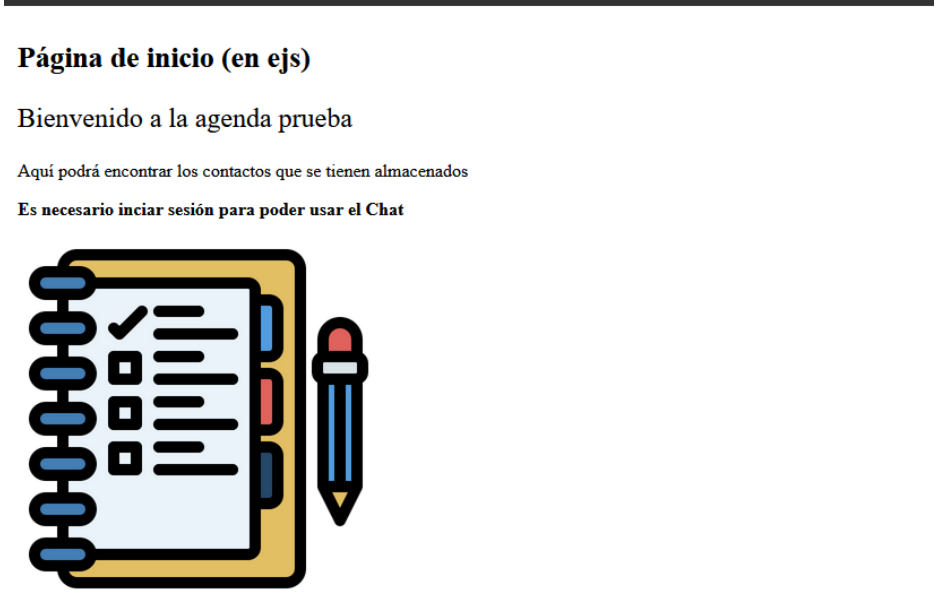
- Clonar el repositorio
- Entrar a la carpeta src (cd src)
- npm i
- npm start
- Acceder a http://localhost:3002/



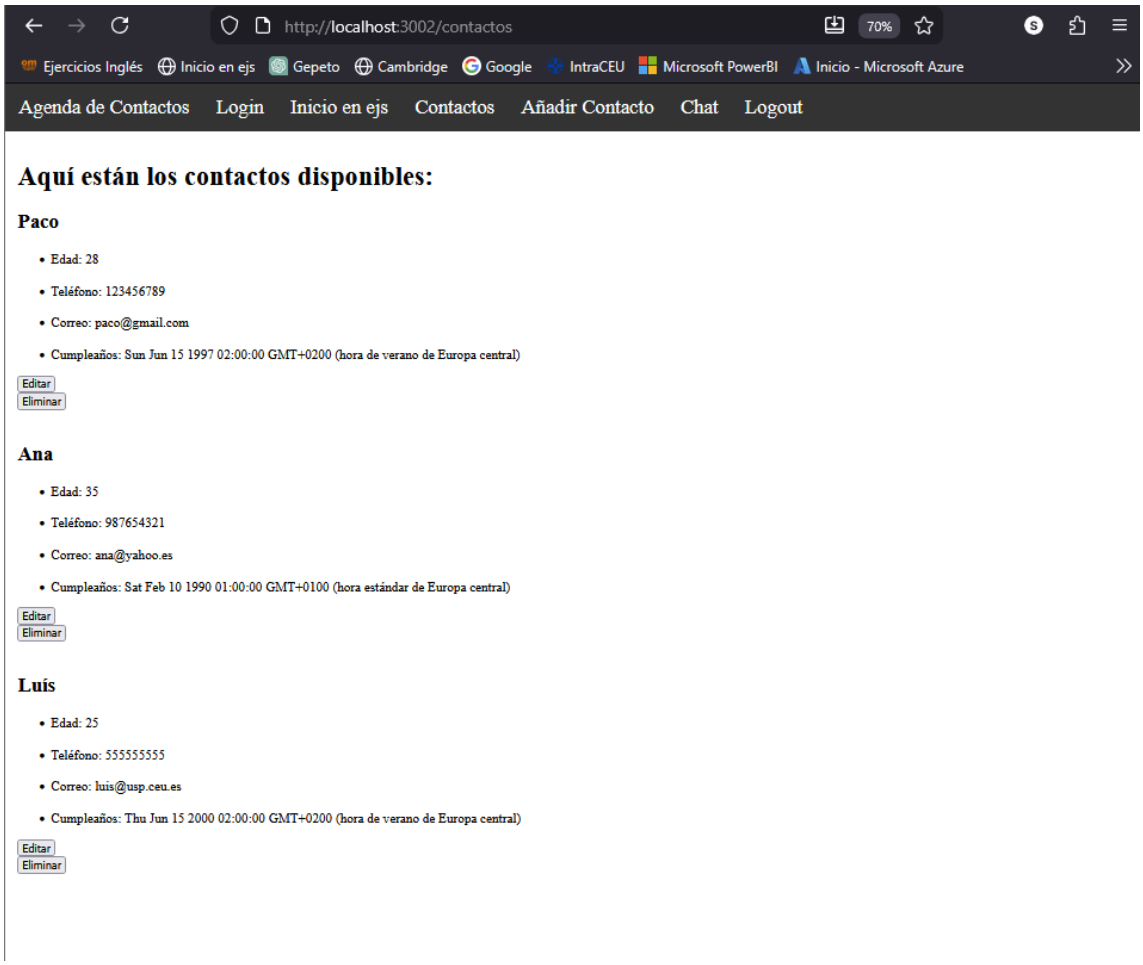
- Botón login para iniciar sesión o registrarse

- Un usuario por defecto por ejemplo es usuario: Ana y contraseña: 1234

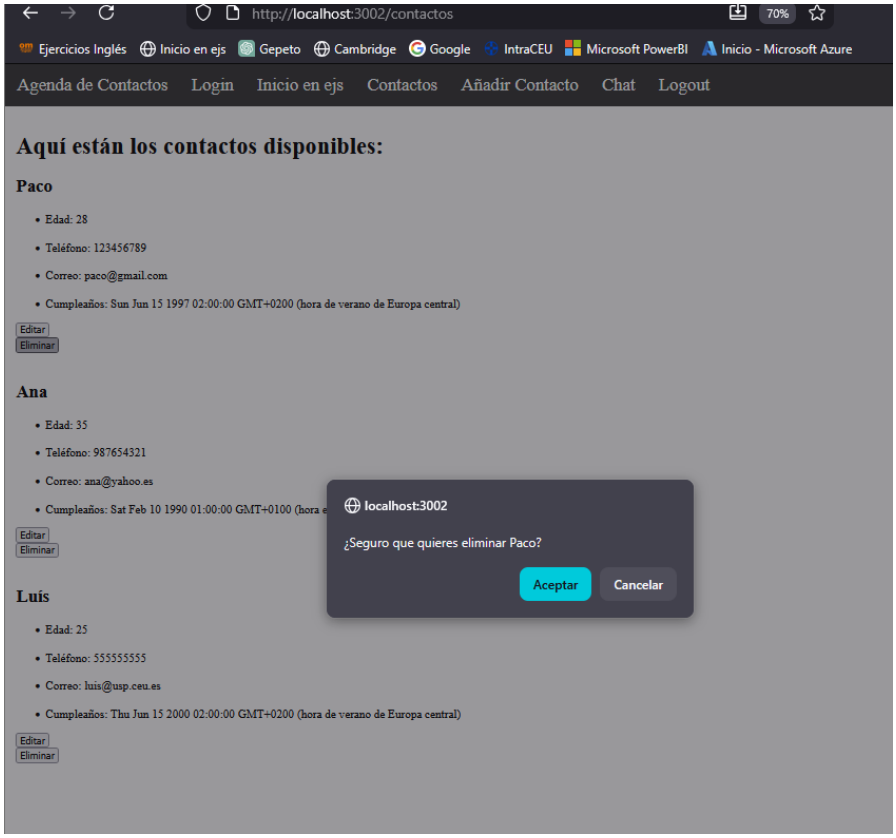
- Si nos queremos registrar, por ejemplo, prueba y 123 de contraseña

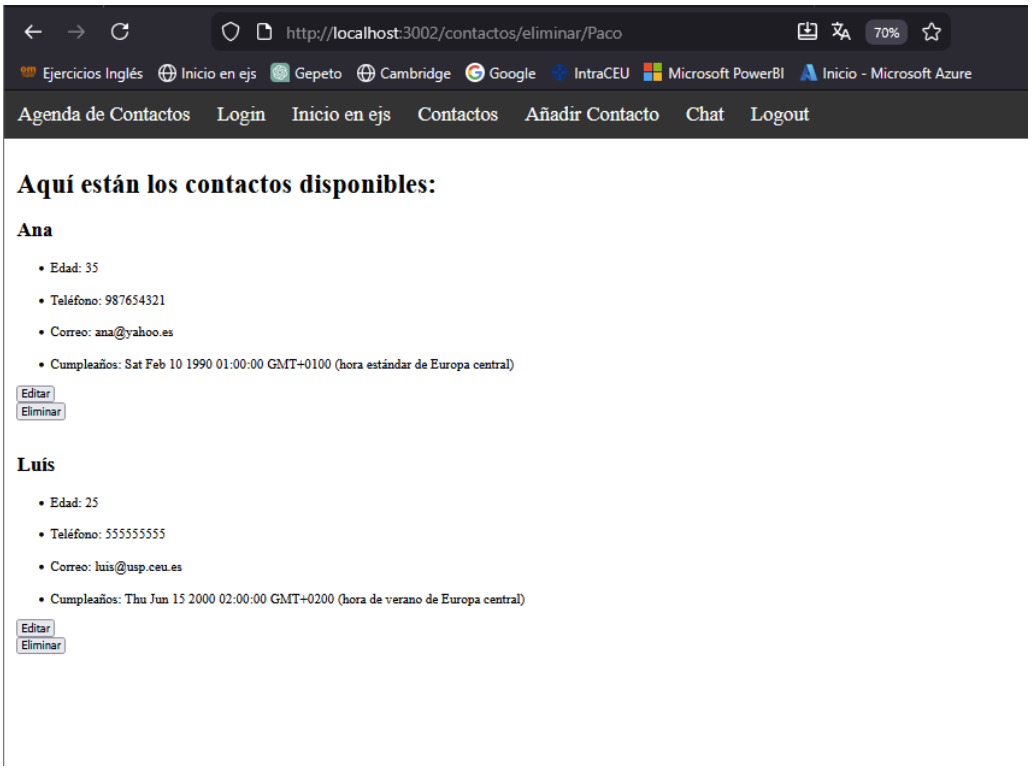


- En contactos veremos los contactos disponibles

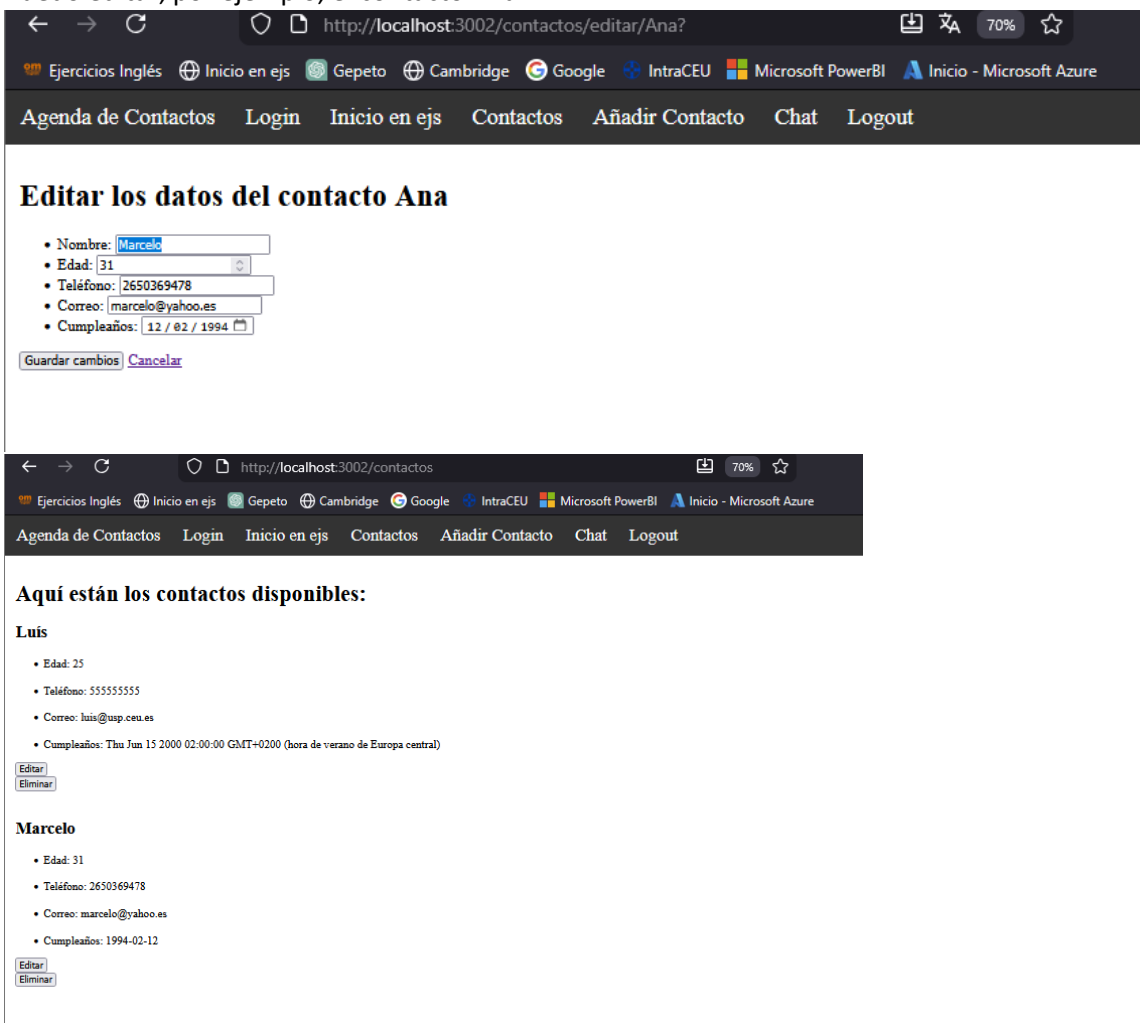


- Estos se pueden editar, eliminar y añadir

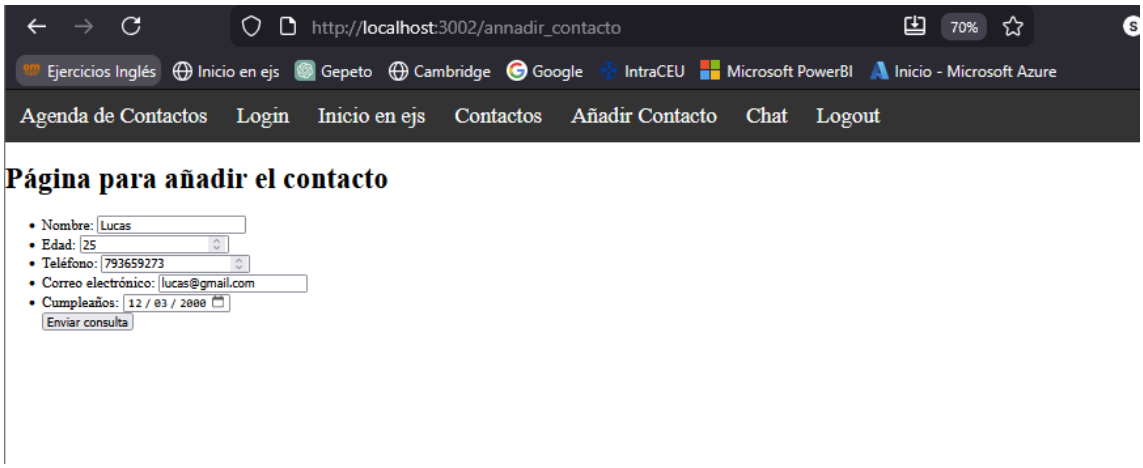


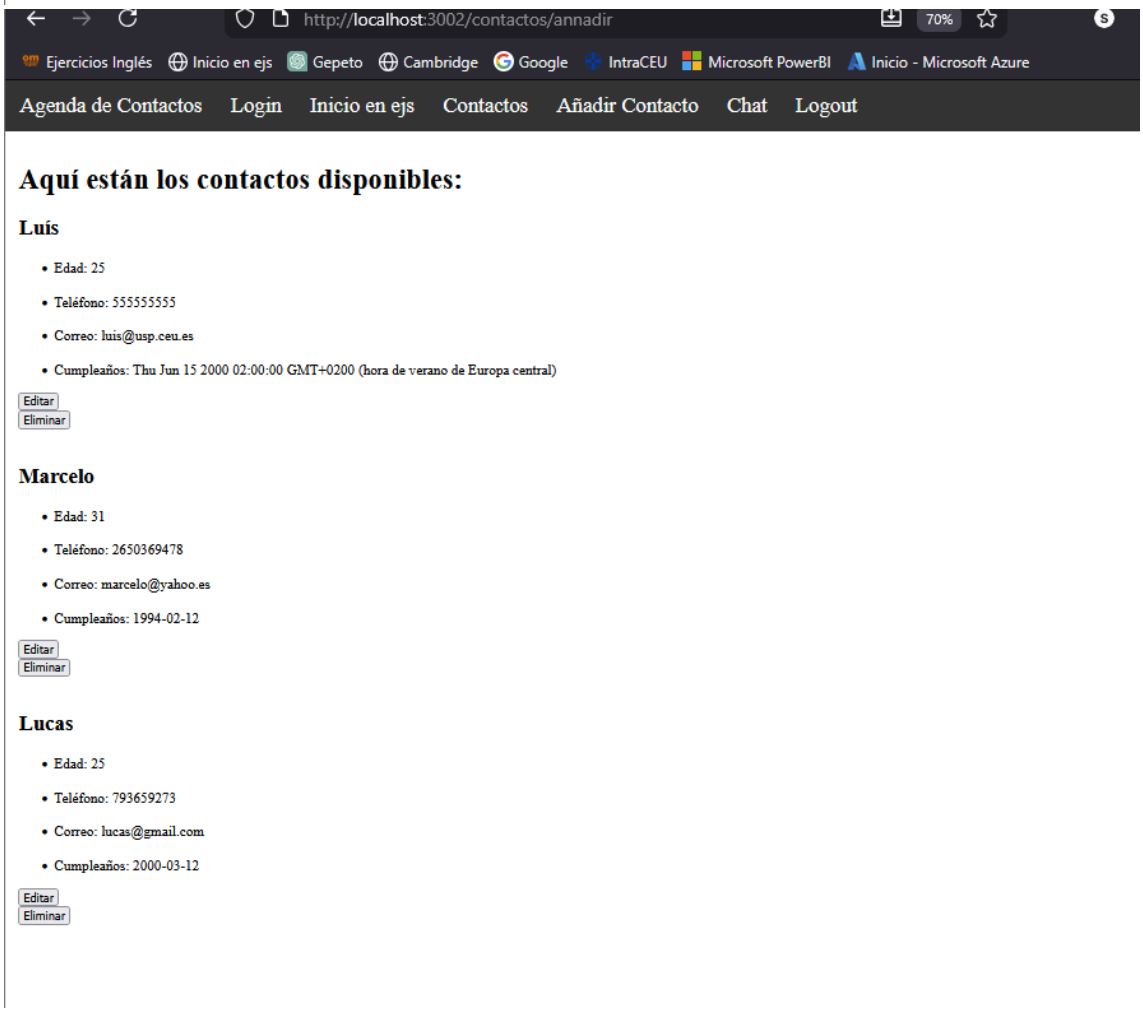


- Puedo editar, por ejemplo, el contacto Ana

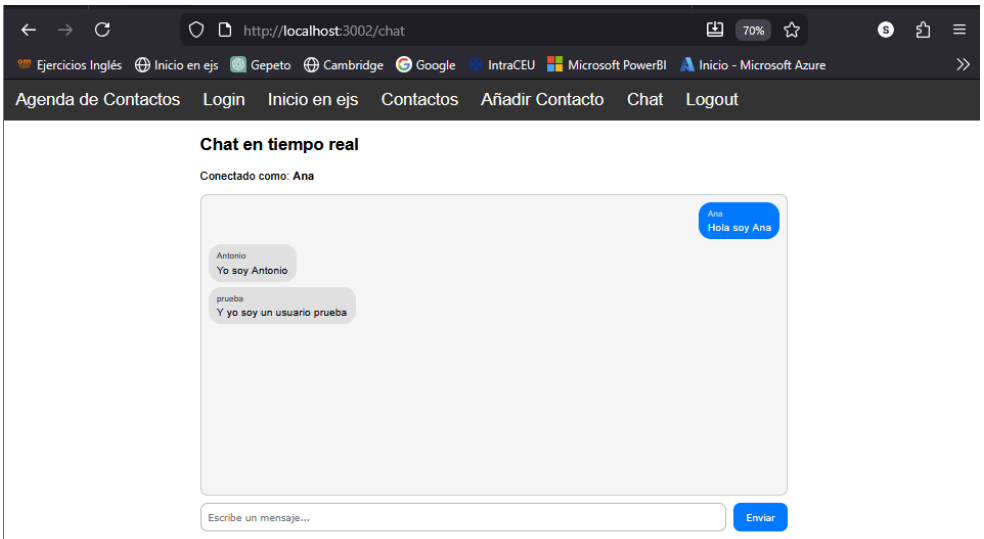


- Puedo añadir contactos

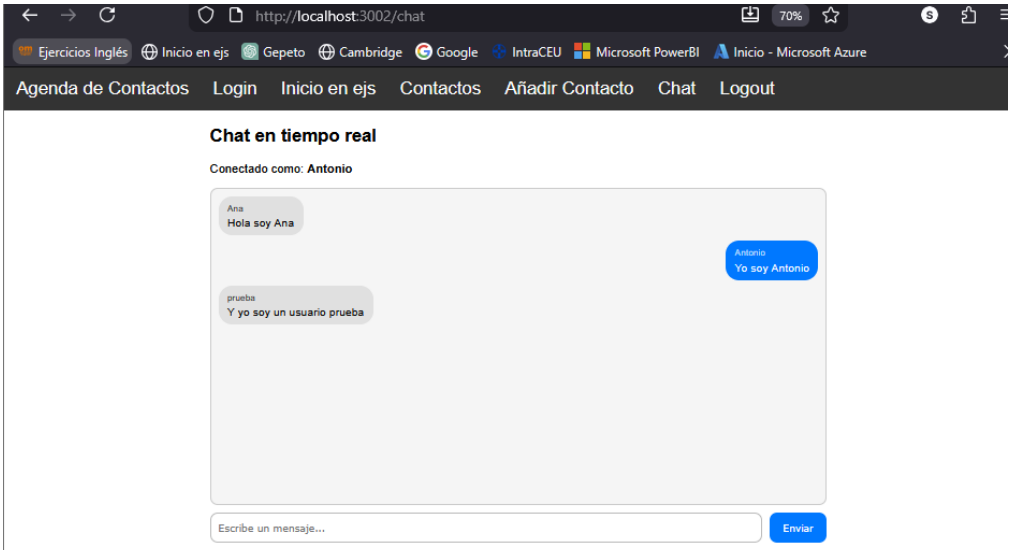




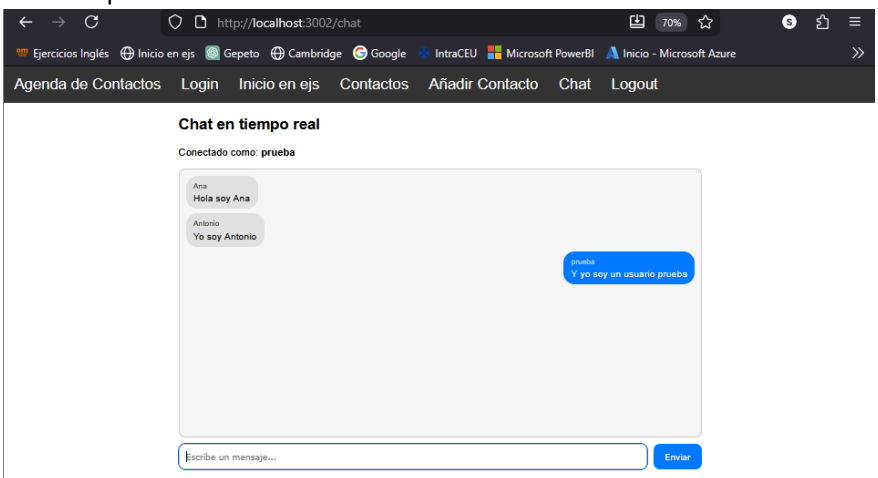
- Puedo acceder al chat, los uso con 3 usuarios a la vez. Ana y Antonio con contraseñas 1234
    - Vista de Ana



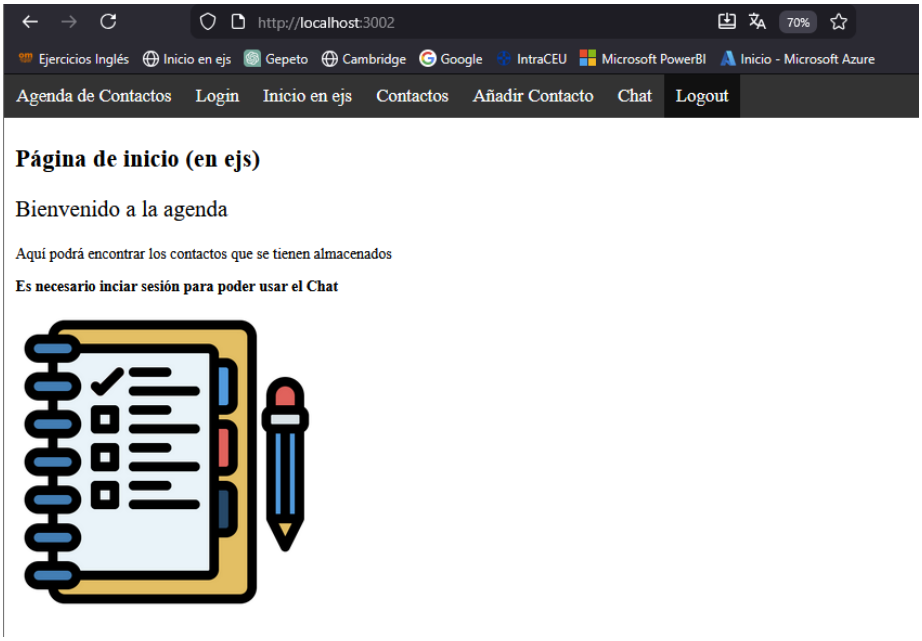
- Vista de Antonio



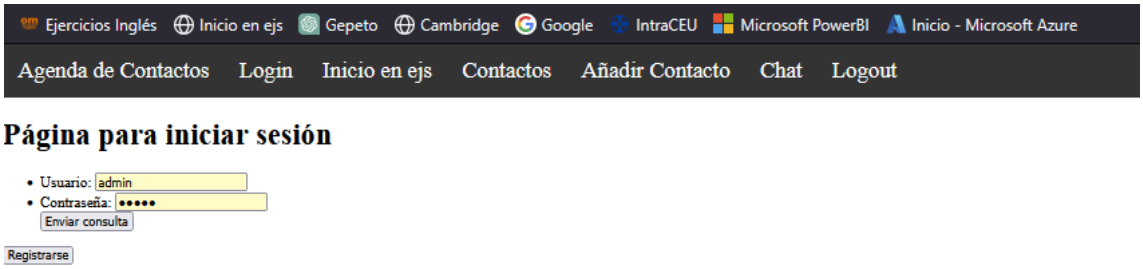
- Vista de prueba



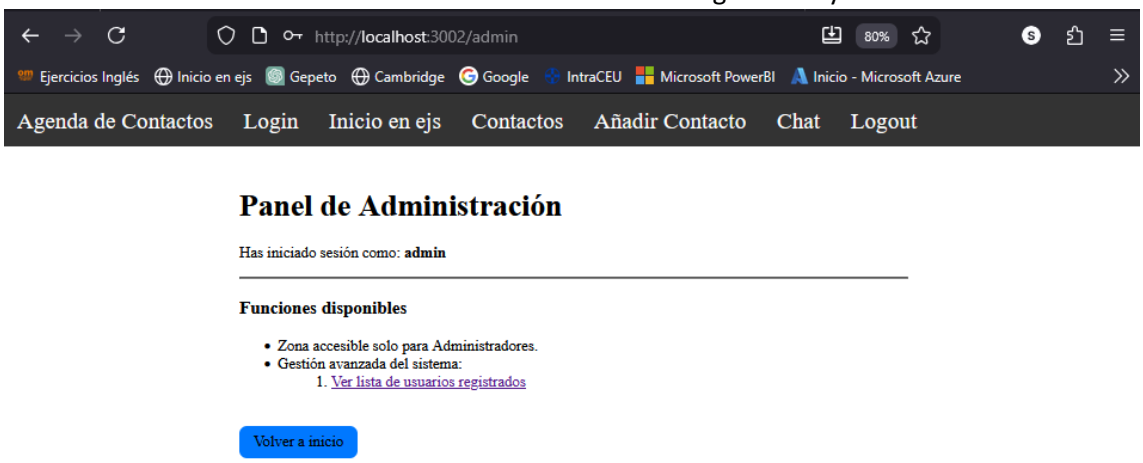
- Puedo hacer logout dándole al botón y este me llevará a Inicio, Como usuario no registrado no puedo acceder al chat

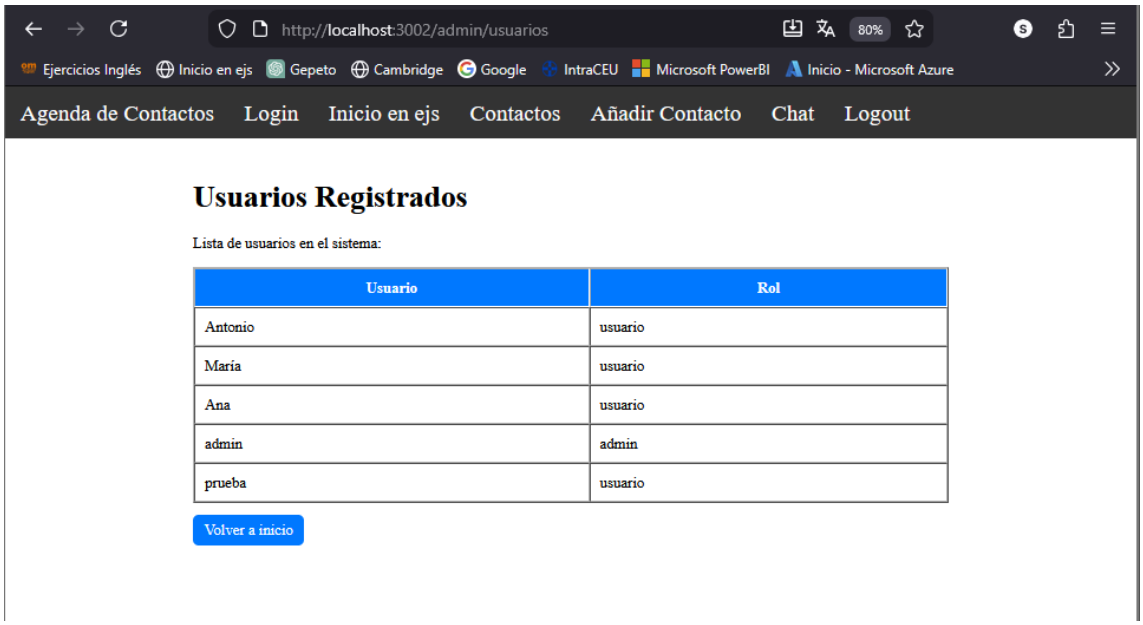


- Para administradores, se usará el usuario admin.



- Este tendrá funcionalidades como ver la lista de usuarios registrados y su rol




http://localhost:3002/admin/usuarios
80%
Ejercicios Inglés
Inicio en ejs
Gepeto
Cambridge
Google
IntraCEU
Microsoft PowerBI
Inicio - Microsoft Azure
Agenda de Contactos
Login
Inicio en ejs
Contactos
Añadir Contacto
Chat
Logout
Usuarios Registrados
Lista de usuarios en el sistema:
Usuario
Rol
Antonio
usuario
María
usuario
Ana
usuario
admin
admin
prueba
usuario
Volver a inicio